

KV-Cache Optimization for LLM Inference

KIVI · TopK · SnapKV · MLA — Llama-2-7B on H100 80GB

Team KV · Rishika Mamidibathula (rm4318), Suhas Morisetty (vm2825), Ziheng Wang (zw3153), Tony Tian (jt3640)

GitHub: github.com/rishika1099/KV-Cache-Implementation

W&B: wandb.ai/rm4318-columbia-university/kv-cache-hpml

Dr. Kaoutar El Maghraoui · Columbia University · HPML Spring 2026

Motivation — The KV Cache Bottleneck

Why KV cache is the bottleneck

- **LLM decoding is memory-bandwidth bound**, since every generated token requires re-reading the KV cache from GPU memory.
- **KV-cache memory grows linearly** with sequence length, batch size, and number of layers.
- **At BS=32 and seq=4K, Llama-2-7B needs ~65 GB just for KV cache**, nearly saturating an H100 80GB GPU.
- **Model weights stay fixed, but KV cache keeps expanding during generation**, creating major memory and latency bottlenecks.

Three compression approaches

- **Quantization**: Fewer bits per KV element (KIVI)
- **Sparse selection**: Attend to only important tokens (TopK, SnapKV)
- **Latent projection**: Low-rank bottleneck representation (MLA)

KV Cache Memory

$$\text{mem} = 2 \times L \times H \times D \times S \times B \times \text{dtype}$$

Symbol	Value	Meaning
L	32	Layers
H	32	Attn heads
D	128	Head dim
S	4096	Seq length
B	32	Batch size
dtype	2 B	FP16
Total	65.5 GB	→ OOM risk

Experimental Setup & Methods Overview

Model & Harness

Llama-2-7b-chat-hf · 32L 32H 128D
PyTorch 2.4.1 · Transformers 4.44
Greedy decode · seed=42 · BS varied

Hardware

1× H100 80GB HBM3 (Modal cloud)
CUDA 12.1 · Triton 3.0 · flash-attn 2.5
Custom KIVI CUDA+Triton kernels

Evaluation

LongBench: 6 tasks × 20 examples
qasper · multifieldqa_en · triviaqa
2wikimqa · multi_news · lcc

Synthetic fixed size batches for uniform
throughput measurements

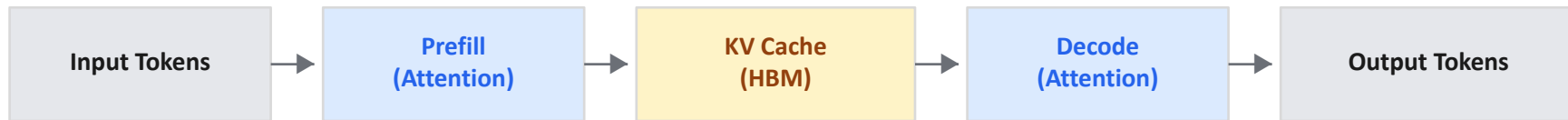
Five Methods at a Glance

Method	Type	Compression	Quality (overall)	Key Tradeoff
Baseline (FP16)	Reference	1×	0.291	—
KIVI 4-bit	Quantize	~4×	0.292 (+0.3%)	Slower at B=1; wins at B≥16
KIVI 2-bit	Quantize	~8×	0.273 (−6.2%)	Best batch throughput; quality cost
TopK K=1024	Sparse	N/A (attn scope)	0.292 (+0.4%)	Per-step scoring overhead at B=1
SnapKV 0.4	Evict	~2.5×	0.295 (+0.3%)	No per-step cost; one-shot eviction
MLA (B=1)	Latent	3.9×	0.081* (−65%)	Needs more training; KV+decode win

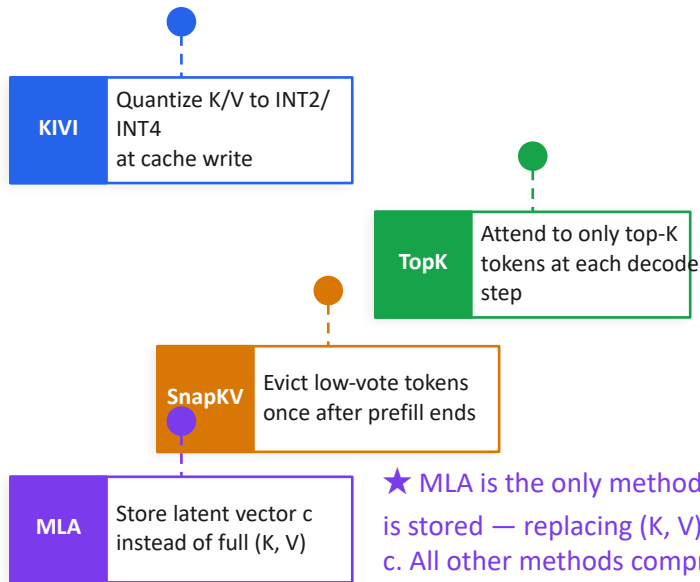
* MLA baseline uses different truncation (2048 vs 4096 tokens) — quality numbers not directly comparable to other methods

Technical Approach

Unified view of the KV cache pipeline and compression points.



Baseline (FP16): no intervention — full K/V retained and read at every decode step.



★ MLA is the only method that fundamentally changes what is stored — replacing (K, V) with a compressed latent vector c . All other methods compress or select from existing K/V tensors.

Profiling Methodology

Tools

- **PyTorch Profiler / torch.profiler**

kernel-level op timing

- **torch.cuda.max_memory_allocated()**

peak GPU memory per run

- **Weights & Biases**

run-level metric logging + comparison

- **Custom timing harness**

TTFT + per-token decode latency

- **Modal volume cache**

model & dataset pinned across runs

Metrics captured

- **TTFT (time to first token)**

prefill latency in ms

- **Decode throughput**

tokens/sec at each batch size

- **Peak GPU memory**

GB via CUDA memory tracker

- **LongBench task score**

F1, ROUGE-L, or edit-similarity per task

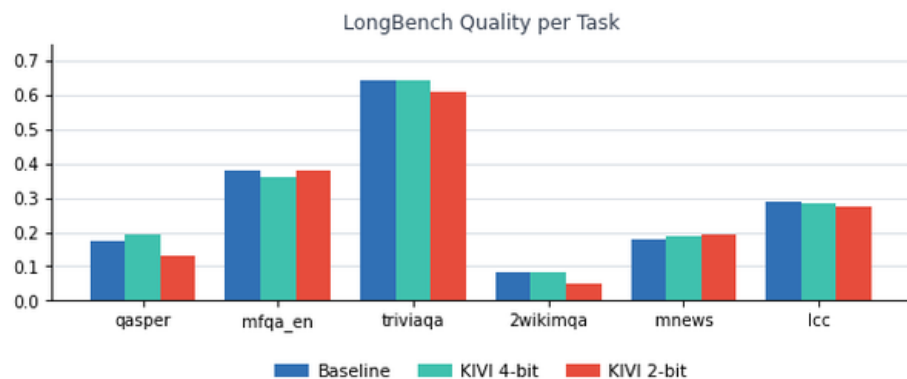
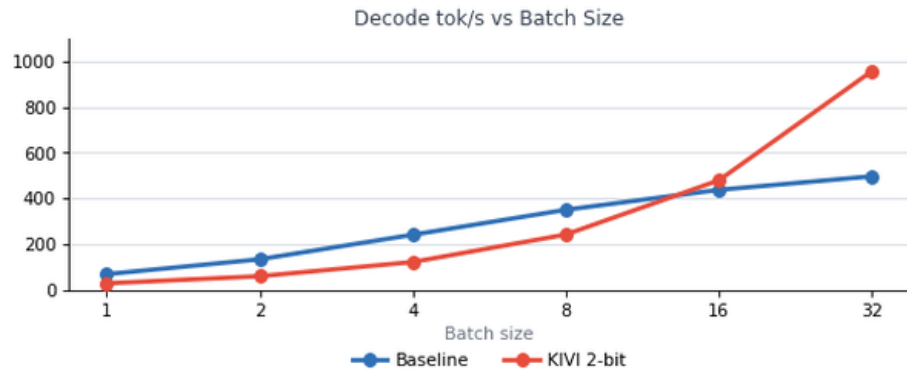
Stability: 5 batches per config, first excluded as warmup; mean of 4 stable batches reported. Model + dataset loaded from pinned Modal volumes for run-to-run reproducibility.

KIVI — 2/4-bit Asymmetric KV Quantization

Design

- **Keys:** per-channel quantization (scale/zero per head across seq dim)
- **Values:** per-token grouped quantization (group_size=32)
- **Residual buffer:** last 128 tokens always FP16; quantized only when full
- **Kernels:** triton_quantize_and_pack fuses INT4 packing; cuda_bmm_fA_qB_outer dequantizes on-the-fly during GEMM
- **Cache format:** 9-tuple per layer (quantized blocks + scales + zeros + residual)

Crossover insight: INT4 GEMV is slower than FP16 GEMV at low batch (GEMV doesn't saturate tensor cores). KIVI wins only once decode becomes near-GEMM at $BS \geq 16$, where HBM bandwidth saving outweighs dequant cost.



TopK / TokenSelect — Sparse Attention

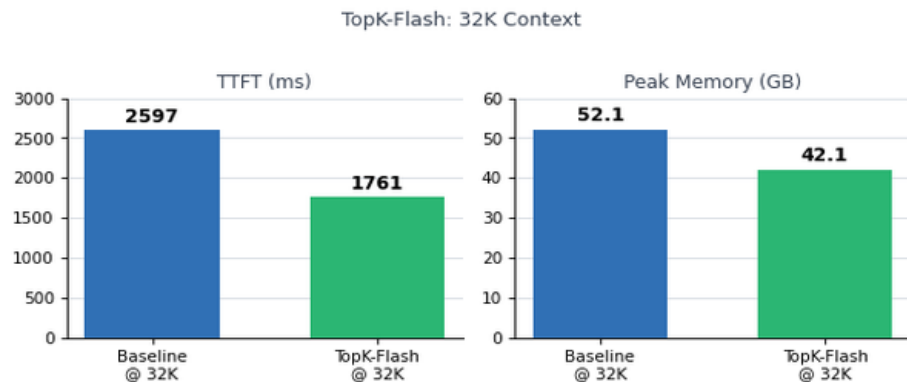
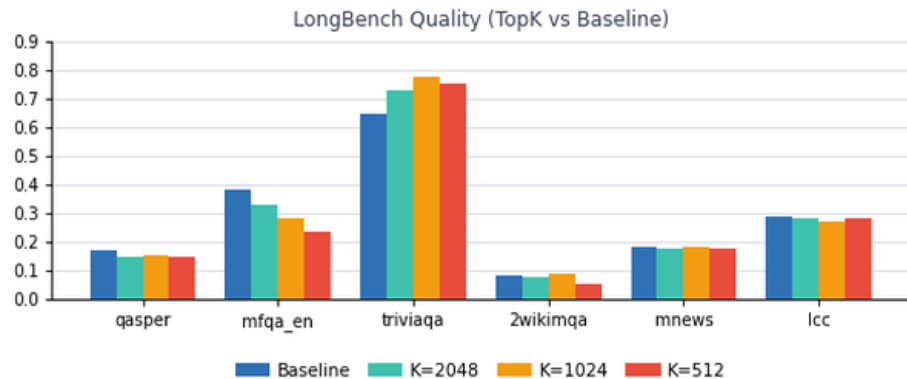
Design

Three-region KV cache (full FP16 retained):

Sink (128) + Top-K middle (dynamic) + Local (512)

- **Scoring:** Triton kernel `_topk_score_kernel` fuses $Q \cdot K^T$ across all 32 heads $\rightarrow$ softmax $\rightarrow$ sum $\rightarrow$ top-K indices
- **Cache reuse:** cosine similarity (threshold=0.9) skips re-scoring if query is similar to previous step
- **RoPE correction:** `true_seq_length` passed to each step to preserve positional encoding after cache slicing

TopK-Flash variant: Paged KV pool (256-token pages) + `flash_attn_with_kvcache` enables zero-copy selective attention. At 32K prefill: TTFT -33%, memory -19% (52 $\rightarrow$ 42 GB). Baseline OOMs at 64K; TopK-Flash handles it at 65.8 GB.

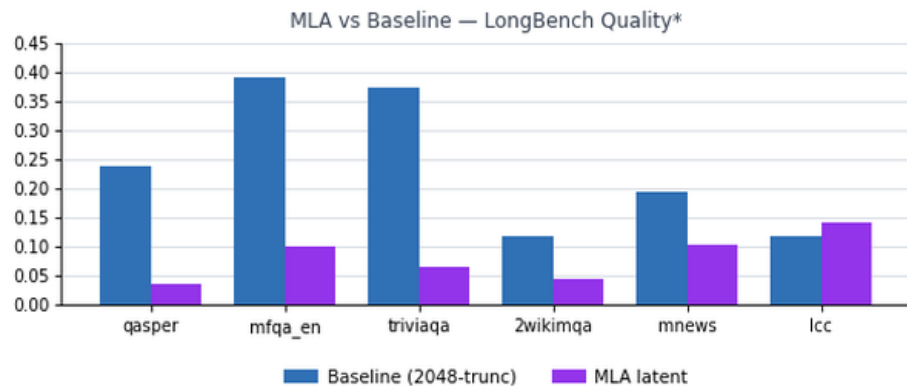
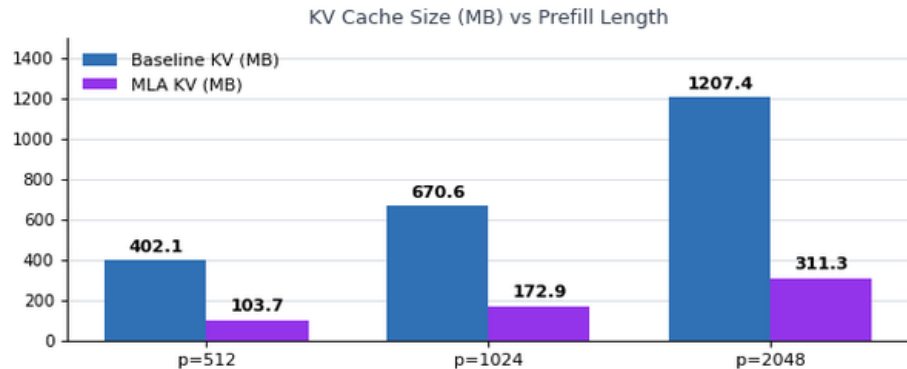


MLA — Multi-head Latent Attention via TransMLA

Design

- **Latent KV:** stores a compressed latent vector c per token instead of full (K,V) tensors; expanded during attention; Attention Absorption speeds up inference.
- **Model:** botsxc/llama2-7b-chat-mla-2048-8 (freqfold-8, 2048 latent dimension, 256 context length calibration), no training
- **KV reduction:** consistently 3.9× smaller across all prefill lengths (402 → 104 MB at $p=512$)
- **Decode speedup:** 1.26–1.32× at $B=1$ — confirms HBM-bandwidth-bound decode even for single-request inference

Quality limitation: Overall drops 0.238 → 0.081 (−65%) on LongBench. QA/retrieval tasks collapse (triviaqa −83%). Only lcc (code) marginally improves. Root cause: checkpoint calibrated at 256-token context with 4x compression and without finetuning — insufficient capacity for long context. Needs finetuning.



* MLA baseline used 2048-token truncation vs 4096 for other methods

SnapKV — One-Shot Prefill Eviction (budget_ratio = 0.4)

Design

How it works (5 steps):

1. Prefill:

run full causal attention over complete prompt

2. Vote: average attention scores over last 32 query positions per head

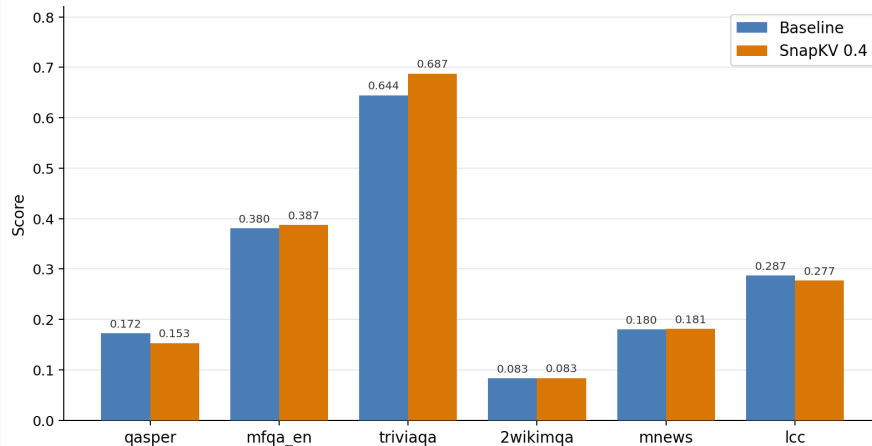
3. Pool: max-pool (kernel_size=7) to smooth noise

4. Select: keep top floor($S \times 0.4$) tokens per head; evict the rest permanently

5. Decode: standard decode over fixed reduced cache — zero per-step overhead

Key insight: triviaqa improves +4.3pt because the observation-window vote actively selects answer-aligned positions and removes irrelevant prefix noise. Only qasper degrades (-2.0pt) where scattered evidence spans fall outside the observation window.

SnapKV vs Baseline — LongBench Quality



Task	Baseline	SnapKV	Δ
triviaqa	0.6444	0.6870	+0.043
multifieldqa	0.3803	0.3870	+0.007
OVERALL	0.2914	0.2946	+0.003
qasper	0.1723	0.1526	-0.020

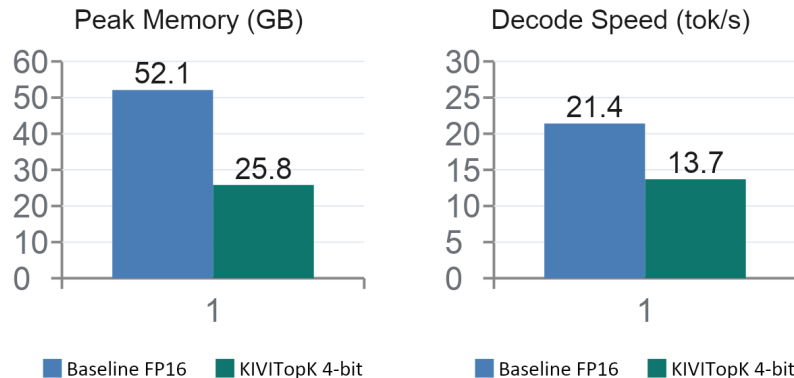
KIVI × TopK — Quantised-Sparse Hybrid (32K context)

How it works

- Prefill:** `F.scaled_dot_product_attention` (flash-attn backed, $O(T)$ memory) — avoids OOM at 32K
- Score:** `cuda_bmm_fA_qB_outer` over all 32K K tokens — unavoidable; need scores before selection
- Select:** `torch.topk` → `n_sink + K + n_local` positions gathered from packed INT4 V rows (64 KB vs 2 MB full)
- Attend:** Gathered INT4 buffer fed directly into `cuda_bmm_fA_qB_outer` — no intermediate FP16 write
- Residual:** FP16 matmul on last 128 tokens + sum — data stays 4-bit through the main kernel

Why decode is still slower (−36%)

- $Q@K^T$ scores all 32K tokens every step — identical cost to baseline (can't skip: need scores to select)
- Extra overhead: `topk` + `sort` + `gather` that baseline never pays



Metric	Baseline FP16	KIVITopK 4-bit	Δ
Decode speed	21.4 tok/s	13.7 tok/s	−36%
Peak GPU mem	52.1 GB	25.8 GB	−51%
LongBench quality	0.291	0.297	+noise

Llama-2-7B · H100 · prefill=32768 · gen=512 · B=1

Tradeoff Analysis — Quality vs Memory vs Throughput

Method	KV Compression	Quality Overall	Decode B=1	Best Batch Tput	Deployment Fit
Baseline	1×	0.291	70 tok/s	497 @ BS=32	Reference
KIVI 4-bit	~4×	0.292 ✓	29 tok/s	873 @ BS=32	Batch serving ≥ BS=16
KIVI 2-bit	~4×	0.273	27 tok/s	957 @ BS=32	Max-batch, accept noise
TopK K=1024	None*	0.292 ✓	43 tok/s	N/A (B=1 only)	Single-req, long-ctx
SnapKV 0.4	~2.5×	0.295 ✓	~70 tok/s †	N/A (B=1 only)	Quality-neutral compress.
MLA (B=1)	3.9×	0.081 ✗	43 tok/s	N/A (B=1 only)	Needs retraining

Key Observations

● No single dominant method — each occupies a distinct quality/memory/latency operating point

● KIVI 4-bit is the headline win for batch serving: lossless quality + 2.3× memory + 1.93× throughput at BS=32

● TopK- Method has additional over because of its sparsity. So benefits in terms of time appear at long context usually from >32K

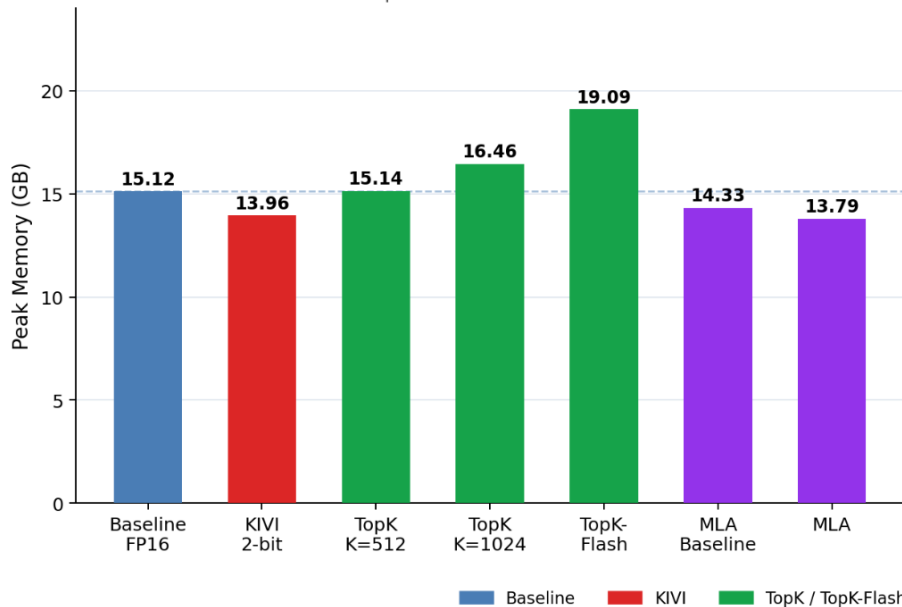
● MLA is a memory and latency win today; quality needs a retrained checkpoint at longer context and higher rank; speed needs absorption implementation

* TopK does not reduce KV storage size — only attention scope is sparse. † SnapKV B=1 throughput interpolated from short-context measurements; not directly benchmarked at long context.

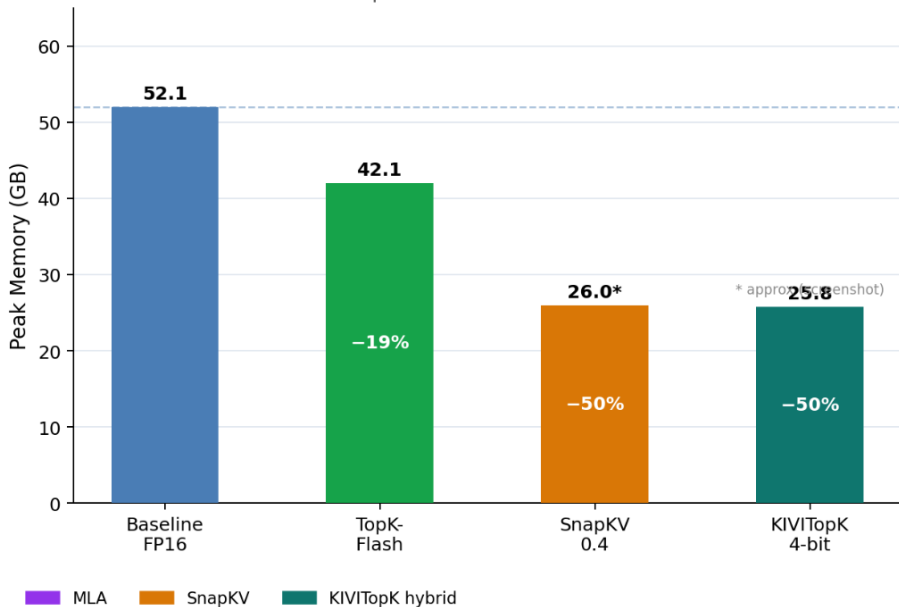
Tradeoff Analysis — Quality vs Memory vs Throughput

Peak GPU Memory at Batch Size = 1 (Llama-2-7B · H100)

prefill = 1024 tokens



prefill = 32 768 tokens



At 1K context, KV compression barely moves the needle — model weights (~14 GB) dominate and all methods cluster within 1.5 GB of each other; the real savings only unlock at 32K+ where the KV cache grows large enough to rival the weights.

Lessons Learned & Future Work

What Worked

- KIVI 4-bit: +0.3% quality, 4× smaller KV, 1.93× throughput at BS=32 — deployable today
- SnapKV 0.4: lossless on 5/6 tasks with zero per-step overhead — surprisingly robust
- TopK K=1024 matches baseline quality via different failure mode (wins triviaqa, loses multifeildqa)
- TopK-Flash paged KV provides real 32K TTFT and memory gains without quality cost

What Didn't Work

- TopK slower at B=1 (43 vs 70 tok/s) — scoring overhead > sparse attention gain at 1K context
- KIVI TTFT 3× slower — prefill quantization overhead is non-negligible for latency-sensitive use
- MLA quality collapse at this checkpoint, batching and absorption needs to be integrated

Future Work that could be explored:

01 Batched decode for TopK

Amortizes per-step scoring cost; expect KIVI-style crossover at BS≥8–16 — paged KV in TopK-Flash already batch-ready

02 MLA checkpoint retraining at 4K+ context

Retrain with r=16–32 latent at 4K–8K context; preserve 3.9× compression while recovering QA task quality

03 KIVI × TopK quantized-sparse hybrid

INT4 storage + sparse attention could yield batch-throughput (KIVI) and long-context TTFT (TopK-Flash) wins simultaneously

Teamwork & Contributions

Member	UNI	Primary Contributions	Key Artifacts
Rishika Mamidibathula	rm4318	TopK + TopK-Flash methods, long-context 32K evaluation, Modal + W&B harness, Code setup, TopK-KIVI hybrid ;	methods/topk_selection.py, llama_topk_flash_model.py, modal_topk_throughput.py,
Suhas Morisetty	vm2825	KIVI 2/4-bit quantization, custom CUDA + Triton kernels, batch-size throughput benchmarking, benchmark setup	methods/kivi_quant.py, llama_kivi_model.py, methods/kivi_kernels/, modal_throughput.py, benchmark/
Ziheng Weng	zw3153	MLA latent KV via TransMLA, MLA throughput sweep, SnapKV integration, RoPE + grouped-query plumbing	methods/transmla/, modal_mla_benchmark.py, modal_mla_throughput.py
Tony Tian	jt3640	SnapKV integration, LongBench experimentation and analysis	methods/snapkv/, evaluation scripts, benchmark integration

Infrastructure & Collaboration

- Unified harness: same model, tokenizer, seed, and hardware across all comparisons
- Modal: reproducible H100 environment pinned via requirements.txt; results persisted to **Modal** volumes; all numbers reproducible
- W&B tracking: all runs logged at wandb.ai/rm4318-columbia-university/kv-cache-hpml — TTFT, decode throughput, peak memory, LongBench per-task scores, method config per run

Thank You

Questions?

GitHub: github.com/rishika1099/KV-Cache-Implementation

W&B: wandb.ai/rm4318-columbia-university/kv-cache-hpml